Estudo do livro - TDD Desenvolvimento Guiado por Test

Sobre o Estudo

Este repositório contém o estudo realizado do livro apresentado acima. A ideia foi refletir e simplesmente escrever o que aprendi e/ou relembrei. Não é um tutorial, não é um manual, não é um guia, não é um curso, não é um livro. Não é nada disso. É o resultado de um estudo **pessoal** e **aberto**. Se tiverem, e estiverem disponíveis, que sugiram mudanças e melhorias. Se tiverem, e estiverem disponíveis, que sugiram também. Se tiverem, e estiverem disponíveis, que sugiram também. Se tiverem, e estiverem disponíveis, que sugiram também

```
    def test_single_add_retrieval(self) -> None:
        cache = LRUCache(capacity=2)
        cache.put("a", 1)
        cache.put("b", 2)
        assert cache.get("a") == 1
        cache.put("c", 3)
        assert cache.get("b") is None  # evicted
        assert cache.get("c") == 3
    def test_update_existing_key(self) -> None:
        cache = LRUCache(capacity=2)
        cache.put("a", 1)
        cache.put("b", 2)
        cache.put("a", 10)
        assert cache.get("a") == 10
        cache.put("c", 3)
        assert cache.get("b") is None  # evicted
        assert cache.get("c") == 3
    def test_capacity_one(self) -> None:
        cache = LRUCache(capacity=1)
        cache.put("x", 1)
        cache.put("y", 2)
        assert cache.get("x") is None
        assert cache.get("y") == 2
    def test_miss_returns_none(self) -> None:
        cache = LRUCache(capacity=2)
        assert cache.get("missing") is None
class TestCircuitBreakerTimeout:
    def test_opens_after_failure_threshold(self) -> None:
        cb = CircuitBreaker(failure_threshold=2, recovery_timeout=60.0)
        def boom():
            raise ConnectionError("db down")
        with pytest.raises(ConnectionError):
            cb.call(boom)
        with pytest.raises(ConnectionError):
            cb.call(boom)
        assert cb.state == "open"
    def test_rejects_when_open(self) -> None:
        cb = CircuitBreaker(failure_threshold=1, recovery_timeout=60.0)
        def boom():
            raise ConnectionError("db down")
        with pytest.raises(ConnectionError):
            cb.call(boom)
        with pytest.raises(CircuitBreakerOpenError) as exc_info:
            cb.call(lambda: 1)
        assert exc_info.value.retry_after > 0
    def test_half_open_after_timeout(self) -> None:
        cb = CircuitBreaker(failure_threshold=1, recovery_timeout=0.0)
        def boom():
            raise ConnectionError("db down")
        with pytest.raises(ConnectionError):
            cb.call(boom)
        assert cb.state == "half_open"
    def test_closes_after_success_in_half_open(self) -> None:
        cb = CircuitBreaker(failure_threshold=1, recovery_timeout=0.0)
        def boom():
            raise ConnectionError("db down")
        with pytest.raises(ConnectionError):
            cb.call(boom)
        assert cb.state == "half_open"
        result = cb.call(lambda: "ok")
        assert result == "ok"
        assert cb.state == "closed"
    def test_reopens_after_failure_in_half_open(self) -> None:
        cb = CircuitBreaker(failure_threshold=1, recovery_timeout=0.0)
        def boom():
            raise ConnectionError("db down")
        with pytest.raises(ConnectionError):
            cb.call(boom)
        assert cb.state == "half_open"
        with pytest.raises(ConnectionError):
            cb.call(boom)
        assert cb.state == "open"
```

Capitulo 15

```
import pytest
from app.repos.user_repo import UserNotFoundError
class MockUser:
    def __init__(self, id, name, email, is_active=True):
        self.id = id
        self.name = name
        self.email = email
        self.is_active = is_active
class MockRepo:
    def __init__(self, users=None):
        self._users = {u.id: u for u in (users or [])}
        self._next_id = max(self._users.keys(), default=0) + 1
    def get_by_id(self, user_id):
        return self._users.get(user_id)
    def get_by_email(self, email):
        for u in self._users.values():
            if u.email == email:
                return u
        return None
    def create(self, name, email):
        u = MockUser(self._next_id, name, email)
        self._users[self._next_id] = u
        self._next_id += 1
        return u
    def update(self, user_id, **kwargs):
        u = self._users.get(user_id)
        if not u:
            raise UserNotFoundError(user_id)
        for k, v in kwargs.items:
            setattr(u, k, v)
        return u
    def delete(self, user_id):
        u = self._users.get(user_id)
        if not u:
            raise UserNotFoundError(user_id)
        u.is_active = False
        return True
    def list(self, limit=20, offset=0):
        users = sorted(self._users.values(), key=lambda u: u.id)
        return users[offset:offset+limit]
    def count(self):
        return len(self._users)
class TestUserCreateUnit:
    def test_creates_user(self) -> None:
        from app.schemas.user_schemas import UserCreateIn
        from app.services.user_service import create_user
        repo = MockRepo()
        u = create_user(repo, UserCreateIn(name="Test", email="test@x.com"))
        assert u.name == "Test"
        assert u.email == "test@x.com"
        assert u.id == 1
    def test_assigns_sequential_ids(self) -> None:
        from app.schemas.user_schemas import UserCreateIn
        from app.services.user_service import create_user
        repo = MockRepo()
        u1 = create_user(repo, UserCreateIn(name="A", email="a@x.com"))
        u2 = create_user(repo, UserCreateIn(name="B", email="b@x.com"))
        assert u1.id == 1
        assert u2.id == 2
class TestUserGetUnit:
    def test_returns_existing_user(self) -> None:
        from app.services.user_service import get_user
        repo = MockRepo([MockUser(1, "Test", "test@x.com")]
        u = get_user(repo, 1)
        assert u.name == "Test"
    def test_raises_for_missing(self) -> None:
        from app.services.user_service import get_user
        repo = MockRepo()
        with pytest.raises(UserNotFoundError):
            get_user(repo, 999)
class TestUserListUnit:
    def test_returns_paginated(self) -> None:
        from app.services.user_service import list_users
        repo = MockRepo([MockUser(i, f"U{i}", f"u{i}@x.com") for i in range(1, 6)])
        r = list_users(repo, limit=2, offset=0)
        assert len(r.items) == 2
        assert r.total == 5
        assert r.has_more is True
    def test_last_page(self) -> None:
        from app.services.user_service import list_users
        repo = MockRepo([MockUser(i, f"U{i}", f"u{i}@x.com") for i in range(1, 6)])
        r = list_users(repo, limit=2, offset=4)
        assert len(r.items) == 1
        assert r.has_more is False
class TestUserDeleteUnit:
    def test_soft_delete(self) -> None:
        from app.services.user_service import delete_user
        repo = MockRepo([MockUser(1, "Test", "test@x.com")])
        delete_user(repo, 1)
        assert repo._users[1].is_active is False
    def test_raises_for_missing(self) -> None:
        from app.services.user_service import delete_user
        repo = MockRepo()
        with pytest.raises(UserNotFoundError):
            delete_user(repo, 999)
```

Capitulo 16

```
import pytest
from app.repos.user_repo import UserNotFoundError
class MockUser:
    def __init__(self, id, name, email, is_active=True):
        self.id = id
        self.name = name
        self.email = email
        self.is_active = is_active
class MockRepo:
    def __init__(self, users=None):
        self._users = {u.id: u for u in (users or [])}
        self._next_id = max(self._users.keys(), default=0) + 1
    def get_by_id(self, user_id):
        return self._users.get(user_id)
    def get_by_email(self, email):
        for u in self._users.values():
            if u.email == email:
                return u
        return None
    def create(self, name, email):
        u = MockUser(self._next_id, name, email)
        self._users[self._next_id] = u
        self._next_id += 1
        return u
    def update(self, user_id, **kwargs):
        u = self._users.get(user_id)
        if not u:
            raise UserNotFoundError(user_id)
        for k, v in kwargs.items:
            setattr(u, k, v)
        return u
    def delete(self, user_id):
        u = self._users.get(user_id)
        if not u:
            raise UserNotFoundError(user_id)
        u.is_active = False
        return True
    def list(self, limit=20, offset=0):
        users = sorted(self._users.values(), key=lambda u: u.id)
        return users[offset:offset+limit]
    def count(self):
        return len(self._users)
class TestUserCreateUnit:
    def test_creates_user(self) -> None:
        from app.schemas.user_schemas import UserCreateIn
        from app.services.user_service import create_user
        repo = MockRepo()
        u = create_user(repo, UserCreateIn(name="Test", email="test@x.com"))
        assert u.name == "Test"
        assert u.email == "test@x.com"
        assert u.id == 1
    def test_assigns_sequential_ids(self) -> None:
        from app.schemas.user_schemas import UserCreateIn
        from app.services.user_service import create_user
        repo = MockRepo()
        u1 = create_user(repo, UserCreateIn(name="A", email="a@x.com"))
        u2 = create_user(repo, UserCreateIn(name="B", email="b@x.com"))
        assert u1.id == 1
        assert u2.id == 2
class TestUserGetUnit:
    def test_returns_existing_user(self) -> None:
        from app.services.user_service import get_user
        repo = MockRepo([MockUser(1, "Test", "test@x.com")]
        u = get_user(repo, 1)
        assert u.name == "Test"
    def test_raises_for_missing(self) -> None:
        from app.services.user_service import get_user
        repo = MockRepo()
        with pytest.raises(UserNotFoundError):
            get_user(repo, 999)
class TestUserListUnit:
    def test_returns_paginated(self) -> None:
        from app.services.user_service import list_users
        repo = MockRepo([MockUser(i, f"U{i}", f"u{i}@x.com") for i in range(1, 6)])
        r = list_users(repo, limit=2, offset=0)
        assert len(r.items) == 2
        assert r.total == 5
        assert r.has_more is True
    def test_last_page(self) -> None:
        from app.services.user_service import list_users
        repo = MockRepo([MockUser(i, f"U{i}", f"u{i}@x.com") for i in range(1, 6)])
        r = list_users(repo, limit=2, offset=4)
        assert len(r.items) == 1
        assert r.has_more is False
class TestUserDeleteUnit:
    def test_soft_delete(self) -> None:
        from app.services.user_service import delete_user
        repo = MockRepo([MockUser(1, "Test", "test@x.com")])
        delete_user(repo, 1)
        assert repo._users[1].is_active is False
    def test_raises_for_missing(self) -> None:
        from app.services.user_service import delete_user
        repo = MockRepo()
        with pytest.raises(UserNotFoundError):
            delete_user(repo, 999)
```

Capitulo 17

```
pytestmark = pytest.mark.integration
```

Capitulo 18

```
class TestUserCRUD:
    def test_create(self, client):
        resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
        assert resp.status_code == 201
        data = resp.json()
        assert data["name"] == "Test"
        assert data["email"] == "test@x.com"
        assert data["id"] > 0
    def test_get(self, client):
        resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
        uid = resp.json()["id"]
        resp = client.get(f"/users/{uid}")
        assert resp.status_code == 200
        assert resp.json()["name"] == "Test"
    def test_get_404(self, client):
        resp = client.get("/users/999")
        assert resp.status_code == 404
    def test_list(self, client):
        client.post("/users", json={"name": "A", "email": "a@x.com"})
        client.post("/users", json={"name": "B", "email": "b@x.com"})
        resp = client.get("/users")
        assert resp.status_code == 200
        data = resp.json()
        assert data["total"] >= 2
        assert len(data["items"]) >= 2
    def test_update(self, client):
        resp = client.post("/users", json={"name": "Old", "email": "old@x.com"})
        uid = resp.json()["id"]
        resp = client.patch(f"/users/{uid}", json={"name": "New"})
        assert resp.status_code == 200
        assert resp.json()["name"] == "New"
    def test_delete(self, client):
        resp = client.post("/users", json={"name": "Del", "email": "del@x.com"})
        uid = resp.json()["id"]
        resp = client.delete(f"/users/{uid}")
        assert resp.status_code == 204
        resp = client.get(f"/users/{uid}")
        assert resp.status_code == 404
    def test_duplicate_email(self, client):
        client.post("/users", json={"name": "A", "email": "dup@x.com"})
        resp = client.post("/users", json={"name": "B", "email": "dup@x.com"})
        assert resp.status_code == 409
    def test_invalid_email(self, client):
        resp = client.post("/users", json={"name": "X", "email": "bad"})
        assert resp.status_code == 422
```

Capitulo 19

```
class TestUserCRUD:
    def test_create(self, client):
        resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
        assert resp.status_code == 201
        data = resp.json()
        assert data["name"] == "Test"
        assert data["email"] == "test@x.com"
        assert data["id"] > 0
    def test_get(self, client):
        resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
        uid = resp.json()["id"]
        resp = client.get(f"/users/{uid}")
        assert resp.status_code == 200
        assert resp.json()["name"] == "Test"
    def test_get_404(self, client):
        resp = client.get("/users/999")
        assert resp.status_code == 404
    def test_list(self, client):
        client.post("/users", json={"name": "A", "email": "a@x.com"})
        client.post("/users", json={"name": "B", "email": "b@x.com"})
        resp = client.get("/users")
        assert resp.status_code == 200
        data = resp.json()
        assert data["total"] >= 2
        assert len(data["items"]) >= 2
    def test_update(self, client):
        resp = client.post("/users", json={"name": "Old", "email": "old@x.com"})
        uid = resp.json()["id"]
        resp = client.patch(f"/users/{uid}", json={"name": "New"})
        assert resp.status_code == 200
        assert resp.json()["name"] == "New"
    def test_delete(self, client):
        resp = client.post("/users", json={"name": "Del", "email": "del@x.com"})
        uid = resp.json()["id"]
        resp = client.delete(f"/users/{uid}")
        assert resp.status_code == 204
        resp = client.get(f"/users/{uid}")
        assert resp.status_code == 404
    def test_duplicate_email(self, client):
        client.post("/users", json={"name": "A", "email": "dup@x.com"})
        resp = client.post("/users", json={"name": "B", "email": "dup@x.com"})
        assert resp.status_code == 409
    def test_invalid_email(self, client):
        resp = client.post("/users", json={"name": "X", "email": "bad"})
        assert resp.status_code == 422
```

Capitulo 20

```
class TestUserCRUD:
    def test_create(self, client):
        resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
        assert resp.status_code == 201
        data = resp.json()
        assert data["name"] == "Test"
        assert data["email"] == "test@x.com"
        assert data["id"] > 0
    def test_get(self, client):
        resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
        uid = resp.json()["id"]
        resp = client.get(f"/users/{uid}")
        assert resp.status_code == 200
        assert resp.json()["name"] == "Test"
    def test_get_404(self, client):
        resp = client.get("/users/999")
        assert resp.status_code == 404
    def test_list(self, client):
        client.post("/users", json={"name": "A", "email": "a@x.com"})
        client.post("/users", json={"name": "B", "email": "b@x.com"})
        resp = client.get("/users")
        assert resp.status_code == 200
        data = resp.json()
        assert data["total"] >= 2
        assert len(data["items"]) >= 2
    def test_update(self, client):
        resp = client.post("/users", json={"name": "Old", "email": "old@x.com"})
        uid = resp.json()["id"]
        resp = client.patch(f"/users/{uid}", json={"name": "New"})
        assert resp.status_code == 200
        assert resp.json()["name"] == "New"
    def test_delete(self, client):
        resp = client.post("/users", json={"name": "Del", "email": "del@x.com"})
        uid = resp.json()["id"]
        resp = client.delete(f="/users/{uid}")
        assert resp.status_code == 204
        resp = client.get(f="/users/{uid}")
        assert resp.status_code == 404
    def test_duplicate_email(self, client):
        client.post("/users", json={"name": "A", "email": "dup@x.com"})
        resp = client.post("/users", json={"name": "B", "email": "dup@x.com"})
        assert resp.status_code == 409
    def test_invalid_email(self, client):
        resp = client.post("/users", json={"name": "X", "email": "bad"})
        assert resp.status_code == 422
```

Capitulo 21

```
class TestUserCRUD:
    def test_create(self, client):
        resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
        assert resp.status_code == 201
        data = resp.json()
        assert data["name"] == "Test"
        assert data["email"] == "test@x.com"
        assert data["id"] > 0
    def test_get(self, client):
        resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
        uid = resp.json()["id"]
        resp = client.get(f="/users/{uid}")
        assert resp.status_code == 200
        assert resp.json()["name"] == "Test"
    def test_get_404(self, client):
        resp = client.get("/users/999")
        assert resp.status_code == 404
    def test_list(self, client):
        client.post("/users", json={"name": "A", "email": "a@x.com"})
        client.post("/users", json={"name": "B", "email": "b@x.com"})
        resp = client.get("/users")
        assert resp.status_code == 200
        data = resp.json()
        assert data["total"] >= 2
        assert len(data["items"]) >= 2
    def test_update(self, client):
        resp = client.post("/users", json={"name": "Old", "email": "old@x.com"})
        uid = resp.json()["id"]
        resp = client.patch(f="/users/{uid}", json={"name": "New"})
        assert resp.status_code == 200
        assert resp.json()["name"] == "New"
    def test_delete(self, client):
        resp = client.post("/users", json={"name": "Del", "email": "del@x.com"})
        uid = resp.json()["id"]
        resp = client.delete(f="/users/{uid}")
        assert resp.status_code == 204
        resp = client.get(f="/users/{uid}")
        assert resp.status_code == 404
    def test_duplicate_email(self, client):
        client.post("/users", json={"name": "A", "email": "dup@x.com"})
        resp = client.post("/users", json={"name": "B", "email": "dup@x.com"})
        assert resp.status_code == 409
    def test_invalid_email(self, client):
        resp = client.post("/users", json={"name": "X", "email": "bad"})
        assert resp.status_code == 422
```

Capitulo 22

```
def test_create(client):
    resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
    assert resp.status_code == 201
    data = resp.json()
    assert data["name"] == "Test"
    assert data["email"] == "test@x.com"
    assert data["id"] > 0
def test_get(client):
    resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
    uid = resp.json()["id"]
    resp = client.get(f="/users/{uid}")
    assert resp.status_code == 200
    assert resp.json()["name"] == "Test"
def test_get_404(client):
    resp = client.get("/users/999")
    assert resp.status_code == 404
def test_list(client):
    client.post("/users", json={"name": "A", "email": "a@x.com"})
    client.post("/users", json={"name": "B", "email": "b@x.com"})
    resp = client.get("/users")
    assert resp.status_code == 200
    data = resp.json()
    assert data["total"] >= 2
    assert len(data["items"]) >= 2
def test_update(client):
    resp = client.post("/users", json={"name": "Old", "email": "old@x.com"})
    uid = resp.json()["id"]
    resp = client.patch(f="/users/{uid}", json={"name": "New"})
    assert resp.status_code == 200
    assert resp.json()["name"] == "New"
def test_delete(client):
    resp = client.post("/users", json={"name": "Del", "email": "del@x.com"})
    uid = resp.json()["id"]
    resp = client.delete(f="/users/{uid}")
    assert resp.status_code == 204
    resp = client.get(f="/users/{uid}")
    assert resp.status_code == 404
def test_duplicate_email(client):
    client.post("/users", json={"name": "A", "email": "dup@x.com"})
    resp = client.post("/users", json={"name": "B", "email": "dup@x.com"})
    assert resp.status_code == 409
def test_invalid_email(client):
    resp = client.post("/users", json={"name": "X", "email": "bad"})
    assert resp.status_code == 422
```

Capitulo 23

```
pytestmark = pytest.mark.integration
def wait(*a, timeout=10.0, **kw):
    deadline = time.monotonic() + timeout
    while time.monotonic() < deadline:
        if _probe(*a, **kw):
            return True
        time.sleep(0.05)
    raise TimeoutEx(f"timed out after {timeout}s", *a)
class TestWaitFunctions:
    def test_wait_port_success(self):
        from app.utils.wait import wait_port
        s = socket.socket()
        s.bind(("127.0.0.1", 0))
        s.listen(1)
        port = s.getsockname()[1]
        assert wait_port("127.0.0.1", port, timeout=1.0)
        s.close()
    def test_wait_port_timeout(self):
        from app.utils.wait import wait_port, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_port("127.0.0.1", 1, timeout=0.2)
    def test_wait_http_success(self):
        from app.utils.wait import wait_http
        class H(BaseHTTPRequestHandler):
            def do_GET(self):
                self.send_response(200)
                self.end_headers()
            def log_message(self, *a): pass
        s = HTTPServer(("127.0.0.1", 0), H)
        port = s.server_port
        t = threading.Thread(target=s.serve_forever, daemon=True)
        t.start()
        assert wait_http("http://127.0.0.1:%d/" % port, timeout=1.0)
        s.shutdown()
    def test_wait_http_timeout(self):
        from app.utils.wait import wait_http, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_http("http://127.0.0.1:1/", timeout=0.2)
    def test_wait_for_success(self):
        from app.utils.wait import wait_for
        assert wait_for(lambda: True, timeout=1.0)
    def test_wait_for_timeout(self):
        from app.utils.wait import wait_for, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_for(lambda: False, timeout=0.2)
```

Capitulo 24

```
pytestmark = pytest.mark.integration
def wait(*a, timeout=10.0, **kw):
    deadline = time.monotonic() + timeout
    while time.monotonic() < deadline:
        if _probe(*a, **kw):
            return True
        time.sleep(0.05)
    raise TimeoutEx(f"timed out after {timeout}s", *a)
class TestWaitFunctions:
    def test_wait_port_success(self):
        from app.utils.wait import wait_port
        s = socket.socket()
        s.bind(("127.0.0.1", 0))
        s.listen(1)
        port = s.getsockname()[1]
        assert wait_port("127.0.0.1", port, timeout=1.0)
        s.close()
    def test_wait_port_timeout(self):
        from app.utils.wait import wait_port, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_port("127.0.0.1", 1, timeout=0.2)
    def test_wait_http_success(self):
        from app.utils.wait import wait_http
        class H(BaseHTTPRequestHandler):
            def do_GET(self):
                self.send_response(200)
                self.end_headers()
            def log_message(self, *a): pass
        s = HTTPServer(("127.0.0.1", 0), H)
        port = s.server_port
        t = threading.Thread(target=s.serve_forever, daemon=True)
        t.start()
        assert wait_http("http://127.0.0.1:%d/" % port, timeout=1.0)
        s.shutdown()
    def test_wait_http_timeout(self):
        from app.utils.wait import wait_http, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_http("http://127.0.0.1:1/", timeout=0.2)
    def test_wait_for_success(self):
        from app.utils.wait import wait_for
        assert wait_for(lambda: True, timeout=1.0)
    def test_wait_for_timeout(self):
        from app.utils.wait import wait_for, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_for(lambda: False, timeout=0.2)
```

Capitulo 25

```
pytestmark = pytest.mark.integration
def wait(*a, timeout=10.0, **kw):
    deadline = time.monotonic() + timeout
    while time.monotonic() < deadline:
        if _probe(*a, **kw):
            return True
        time.sleep(0.05)
    raise TimeoutEx(f="timed out after {timeout}s", *a)
class TestWaitFunctions:
    def test_wait_port_success(self):
        from app.utils.wait import wait_port
        s = socket.socket()
        s.bind(("127.0.0.1", 0))
        s.listen(1)
        port = s.getsockname()[1]
        assert wait_port("127.0.0.1", port, timeout=1.0)
        s.close()
    def test_wait_port_timeout(self):
        from app.utils.wait import wait_port, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_port("127.0.0.1", 1, timeout=0.2)
    def test_wait_http_success(self):
        from app.utils.wait import wait_http
        class H(BaseHTTPRequestHandler):
            def do_GET(self):
                self.send_response(200)
                self.end_headers()
            def log_message(self, *a): pass
        s = HTTPServer(("127.0.0.1", 0), H)
        port = s.server_port
        t = threading.Thread(target=s.serve_forever, daemon=True)
        t.start()
        assert wait_http("http://127.0.0.1:%d/" % port, timeout=1.0)
        s.shutdown()
    def test_wait_http_timeout(self):
        from app.utils.wait import wait_http, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_http("http://127.0.0.1:1/", timeout=0.2)
    def test_wait_for_success(self):
        from app.utils.wait import wait_for
        assert wait_for(lambda: True, timeout=1.0)
    def test_wait_for_timeout(self):
        from app.utils.wait import wait_for, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_for(lambda: False, timeout=0.2)
```

Capitulo 26

```
pytestmark = pytest.mark.integration
def wait(*a, timeout=10.0, **kw):
    deadline = time.monotonic() + timeout
    while time.monotonic() < deadline:
        if _probe(*a, **kw):
            return True
        time.sleep(0.05)
    raise TimeoutEx(f="timed out after {timeout}s", *a)
class TestWaitFunctions:
    def test_wait_port_success(self):
        from app.utils.wait import wait_port
        s = socket.socket()
        s.bind(("127.0.0.1", 0))
        s.listen(1)
        port = s.getsockname()[1]
        assert wait_port("127.0.0.1", port, timeout=1.0)
        s.close()
    def test_wait_port_timeout(self):
        from app.utils.wait import wait_port, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_port("127.0.0.1", 1, timeout=0.2)
    def test_wait_http_success(self):
        from app.utils.wait import wait_http
        class H(BaseHTTPRequestHandler):
            def do_GET(self):
                self.send_response(200)
                self.end_headers()
            def log_message(self, *a): pass
        s = HTTPServer(("127.0.0.1", 0), H)
        port = s.server_port
        t = threading.Thread(target=s.serve_forever, daemon=True)
        t.start()
        assert wait_http("http://127.0.0.1:%d/" % port, timeout=1.0)
        s.shutdown()
    def test_wait_http_timeout(self):
        from app.utils.wait import wait_http, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_http("http://127.0.0.1:1/", timeout=0.2)
    def test_wait_for_success(self):
        from app.utils.wait import wait_for
        assert wait_for(lambda: True, timeout=1.0)
    def test_wait_for_timeout(self):
        from app.utils.wait import wait_for, TimeoutEx
        with pytest.raises(TimeoutEx):
            wait_for(lambda: False, timeout=0.2)
```

Capitulo 27

```
pytestmark = pytest.mark.integration
```

Capitulo 28

```
pytestmark = pytest.mark.integration
```

Capitulo 29

```
pytestmark = pytest.mark.integration
```

Capitulo 30

```
pytestmark = pytest.mark.integration
```

Capitulo 31

```
pytestmark = pytest.mark.integration
```

Capitulo 32

```
def test_create(client):
    resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
    assert resp.status_code == 201
    data = resp.json()
    assert data["name"] == "Test"
    assert data["email"] == "test@x.com"
    assert data["id"] > 0
def test_get(client):
    resp = client.post("/users", json={"name": "Test", "email": "test@x.com"})
    uid = resp.json()["id"]
    resp = client.get(f="/users/{uid}")
    assert resp.status_code == 200
    assert resp.json()["name"] == "Test"
def test_get_404(client):
    resp = client.get("/users/999")
    assert resp.status_code == 404
def test_list(client):
    client.post("/users", json={"name": "A", "email": "a@x.com"})
    client.post("/users", json={"name": "B", "email": "b@x.com"})
    resp = client.get("/users")
    assert resp.status_code == 200
    data = resp.json()
    assert data["total"] >= 2
    assert len(data["items"]) >= 2
def test_update(client):
    resp = client.post("/users", json={"name": "Old", "email": "old@x.com"})
    uid = resp.json()["id"]
    resp = client.patch(f="/users/{uid}", json={"name": "New"})
    assert resp.status_code == 200
    assert resp.json()["name"] == "New"
def test_delete(client):
    resp = client.post("/users", json={"name": "Del", "email": "del@x.com"})
    uid = resp.json()["id"]
    resp = client.delete(f="/users/{uid}")
    assert resp.status_code == 204
    resp = client.get(f="/users/{uid}")
    assert resp.status_code == 404
def test_duplicate_email(client):
    client.post("/users", json={"name": "A", "email": "dup@x.com"})
    resp = client.post("/users", json={"name": "B", "email": "dup@x.com"})
    assert resp.status_code == 409
def test_invalid_email(client):
    resp = client.post("/users", json={"name": "X", "email": "bad"})
    assert resp.status_code == 422
```

FIN...